Adding Pollinator Routes and Managing Images

Developer reference for adding new pollinator routes, updating seed data, managing the spritesheet, and re-seeding the database.

Table of Contents

---

1. Data Architecture

All route and pollinator data originates from two seed files that are loaded into the database at startup.

- `backend/prisma/seed.js` — MySQL/MariaDB seed (primary)
- `backend/prisma/seed.pg.js` — PostgreSQL seed (must stay identical in content)

Always edit both seed files together. The only difference between them is apostrophe encoding in string literals (MySQL uses `\'`, PostgreSQL uses `''`). Content must otherwise be identical.

The backend is the sole source of truth for route data. `POST /api/get-pollinator-names` returns all pollinator names and spritesheet coordinates; the frontend uses this endpoint instead of any static JSON file. `shared/data/Routes.json` has been removed — see §8.

---

2. Seed File Structure

Each seed file defines three top-level constants.

`ROUTE_NODES`

A decision-tree node lookup. Keys are `"ROW"` (for the root) or `"ROW.COL"` (for all other nodes). Each node has a `text` label displayed on the route card.

```
const ROUTE_NODES = {
  "0": { text: "Start" },
  "1": {
    "0": { text: "Two Legs" },
    "1": { text: "Six Legs" },
  },
  "2": {
    "0": { text: "Mammal" },
    "1": { text: "Wings" },
    "2": { text: "No Wings" },
    // ...
  },
```

```
  // ...
  "5": {
    "0": { text: "Bat" },
    "1": { text: "Moth" },
    "2": { text: "Butterfly" },
    "3": { text: "Fly" },
  },
  // ...
};
```

The final row nodes (leaf nodes) are the pollinator reveal stops. The node key for a leaf is the same string used as the spritesheet coordinate for that pollinator's image (see §3).

`ROUTES`

Maps pollinator name → ordered array of node keys, start to finish. The last key is always the leaf/reveal node.

```
const ROUTES = {
  Human:       ["0", "1.0", "2.0", "3.0", "4.0"],
  Bat:         ["0", "1.0", "2.0", "3.1", "4.1", "5.0"],
  Hummingbird: ["0", "1.0", "2.1", "3.2", "4.2"],
  // ...
  Bee:         ["0", "1.1", "2.1", "3.3", "4.5", "5.4", "6.0"],
  Wasp:        ["0", "1.1", "2.1", "3.3", "4.5", "5.4", "6.1"],
};
```

`FACTS`

Maps pollinator name → ordered array of fact strings shown on the fact cards during the route.

```
const FACTS = {
  Bat: [
    "Do you like eating bananas and mangos? Bats help pollinate these fruits and
more.",
    "It is believed that bats play a role in pollinating more than 500 types of
tropical plants.",
    // ...
  ],
  // ...
};
```

---

3. Spritesheet and Image System

All illustrations are packed into a single PNG spritesheet:
`frontend/public/images/spritesheet_transparent.png`

Each cell is **100 × 100 px**. The grid is addressed by row (Y) and column (X), both zero-indexed from the top-left.

How the frontend reads coordinates

The `PollinatorImage` component in `frontend/src/app/components.tsx` receives a `coords` string (e.g. `"5.0"`) and computes CSS `backgroundPosition`:

```
const split = coords.split(".");
let xCoord = parseInt(split[1]) * -100;
let yCoord = parseInt(split[0]) * -100;
if (yCoord === 0) xCoord = 0;   // row 0 special case: always column 0
```

So `"5.0"` → `backgroundPosition: "0px -500px"` (left edge, 500px down).

The reveal node key stored in `ROUTES` doubles as the spritesheet coordinate. For example, the last element of `Bat: ["0", ..., "5.0"]` is `"5.0"`, which the frontend uses directly to locate the Bat sprite at row 5, column 0.

Current pollinator sprite map

Pollinator	Node key / sprite coord	Row	Col	Top offset	Left offset
Human	`4.0`	4	0	400px	0px
Hummingbird	`4.2`	4	2	400px	200px
Sunbird	`4.3`	4	3	400px	300px
Beetle	`4.8`	4	8	400px	800px
Bat	`5.0`	5	0	500px	0px
Moth	`5.1`	5	1	500px	100px
Butterfly	`5.2`	5	2	500px	200px
Fly	`5.3`	5	3	500px	300px
Ant	`3.4`	3	4	300px	400px
Bee	`6.0`	6	0	600px	0px
Wasp	`6.1`	6	1	600px	100px

Silhouette rendering

Undiscovered pollinators in the collection screen are rendered using `filter: brightness(0) saturate(100%)`. This works correctly only when the sprite has a **transparent background** — always export as PNG with transparency, never JPEG or PNG with a white fill.

---

4. Adding a New Pollinator Route

Work through the steps in order. All code examples use "Dragonfly" with reveal node `"6.2"` as a running example.

Step 1 — Choose a reveal node key

Pick a `"ROW.COL"` key not already used by any leaf node (see the sprite map above). New pollinators are typically added as the next column in the deepest occupied row, or as a new row if the existing rows are full.

Example: row 6 currently has `6.0` and `6.1`, so the next available key is `"6.2"`.

Step 2 — Add the leaf node to `ROUTE_NODES` (both seed files)

Find the row object in `ROUTE_NODES` and add the new column:

```
// Before
"6": { "0": { text: "Bee" }, "1": { text: "Wasp" } },

// After
"6": { "0": { text: "Bee" }, "1": { text: "Wasp" }, "2": { text: "Dragonfly" } },
```

If the pollinator needs a new row entirely:

```
"7": { "0": { text: "Dragonfly" } },
```

Only add nodes that do not already exist. Intermediate branch nodes (e.g. "Six Legs", "Wings") are shared across routes — do not duplicate them.

Step 3 — Add the route path to `ROUTES` (both seed files)

```
const ROUTES = {
  // ... existing routes ...
  Dragonfly: ["0", "1.1", "2.1", "3.3", "4.5", "5.4", "6.2"],
};
```

Requirements:

- First element must be `"0"` (the shared root node)
- Last element must be the reveal node key from Step 1
- Every element must exist as a key in `ROUTE_NODES`
- Name is case-sensitive and must be consistent across all three files

Step 4 — Add facts to `FACTS` (both seed files)

```
const FACTS = {
  // ... existing facts ...
  Dragonfly: [
    "Dragonflies are among the oldest pollinators, with fossils dating back 300
million years.",
    "They can hover in place and fly backwards — useful for visiting flowers from
any angle.",
    "A dragonfly's compound eyes give it nearly 360-degree vision.",
  ],
};
```

Step 5 — Add the sprite to the spritesheet

1. Create the illustration: **100 × 100 px, PNG, transparent background**
2. Calculate the pixel position of the reveal node cell:

```
top  = row × 100      →  6 × 100 = 600px
left = col × 100      →  2 × 100 = 200px
```

3. Open `frontend/public/images/spritesheet_transparent.png` in an image editor
4. If the new cell is outside the current canvas bounds, extend the canvas — do not resize or move existing content
5. Place the new illustration at the calculated position
6. Export as PNG with transparency, replacing the existing file

Step 6 — Re-seed the database

See §7.

---

5. Updating a Pollinator's Image

No seed changes are required for an image-only replacement — the node key and coordinate stay the same.

1. Look up the sprite coordinate from the table in §3
2. Calculate pixel position: `top = row × 100`, `left = col × 100`
3. Open `frontend/public/images/spritesheet_transparent.png`
4. Replace the 100 × 100 px cell at that position with the new artwork
5. Keep all other cells unchanged
6. Export as PNG with transparency, replacing the existing file
7. Redeploy the frontend — no database operation needed

---

6. Changing Facts for an Existing Route

Edit the relevant pollinator's array in `const FACTS` in both seed files. The format is:

```
Bat: [
  "Fact one.",
  "Fact two.",
  "Fact three.",
],
```

- Facts display in the order they appear in the array
- Each string is a single fact card
- After editing both seed files, re-seed the database (§7)

---

7. Re-seeding the Database

The seed script uses Prisma upserts for all route data, so it is safe to run against a live database with existing player data. Player sessions and route cycle records are not touched.

Development (Docker)

```
# MySQL/MariaDB dev stack
docker compose -f docker-compose.dev.yml exec backend npx prisma db seed

# PostgreSQL dev stack
docker compose -f docker-compose.dev.pg.yml exec backend npx prisma db seed
```

Or restart the migrate service, which runs the seed as part of its startup:

```
docker compose -f docker-compose.dev.pg.yml restart migrate
```

Production

The `migrate` service in the production compose file runs `prisma migrate deploy && node prisma/seed.pg.js` on startup. To re-seed without a full restart:

```
docker compose -f docker-compose.prod.pg.yml exec migrate node prisma/seed.pg.js
```

If the `migrate` container has exited (it is a one-shot service), run the seed in the `backend` container instead:

```
docker compose -f docker-compose.prod.pg.yml exec backend node prisma/seed.pg.js
```

Verification

After seeding, confirm the new route loaded:

```
docker compose -f docker-compose.prod.pg.yml exec postgres \
  psql -U <DB_USER> -d <DB_NAME> -c \
  "SELECT id, name FROM \"Route\" ORDER BY id DESC LIMIT 5;"
```

---

8. Routes.json — Removed

`shared/data/Routes.json` has been deleted. The migration is complete.

What replaced it: `POST /api/get-pollinator-names` returns all pollinator names and spritesheet coordinates directly from the database. Both frontend pages that previously read `Routes.json` now call this endpoint:

- `frontend/src/app/home/page.tsx` — calls `apiFetchAuthed("/api/get-pollinator-names")` to get the total pollinator count for the "X / 11" display
- `frontend/src/app/pollinator-collection/page.tsx` — calls `apiFetchAuthed("/api/get-pollinator-names")` to enumerate pollinators and resolve their silhouette sprite coords

When adding a new pollinator: only update the seed files (Steps 1–4) and the spritesheet (Step 5). There is no JSON file to maintain. The API endpoint automatically includes the new pollinator after the database is re-seeded (Step 6).

This file is not tracked in version control — update it when the route structure, image system, or re-seed process changes.